实验3

软件架构与界面设计

AI象棋对弈助手系统的设计与实现

学号：24111302009

姓名：闫俊

需求输入：实验2 V1.0

设计版本：V1.0

2026年9月

包含架构、数据、接口、流程、界面原型与仓库提交说明

学号：24111302009

姓名：闫俊

个人软件题目：AI象棋对弈助手系统的设计与实现

需求输入：实验2《个人软件选题与需求分析》V1.0

设计版本：V1.0

提交日期：2026年9月

> 本报告以实验2的“摆局—合法性检查—人机对弈—解释—复盘”核心流程为设计基线。软件目标是后续打包为 Windows 桌面 App；本阶段只完成架构、数据、接口、流程和界面设计，不把设计稿误写成已完成的软件功能。

一、设计范围与核心流程

本次实验只设计一条可独立交付的核心流程：用户从首页进入新对弈或摆放局面，系统校验局面和走法，调用本地中国象棋引擎返回候选着法，展示评价与简短解释，对局结束后生成关键节点复盘和练习建议。

本阶段纳入：桌面入口、棋盘交互、规则校验、引擎适配器、结构化解释、复盘摘要、本地 JSON 保存、错误与人工确认状态。

本阶段不纳入：联网匹配、账号/排行榜、社交系统、云端棋谱社区、语音课程和移动端适配。

二、2.1 需求输入与设计追踪

2.1.1 需求—设计对应表

需求编号	实验2来源	本次准备完成的设计	对应图/文件	验收关联
US-01	快速开始人机对弈	首页、对弈页、应用服务层	图5、图2、原型图2	AC-01
US-02	提示非法走法和非法局面	规则校验模块、错误状态	图2、图4	AC-02
US-03	查看候选着法和解释	EngineAdapter、解释适配器、解释区域	图2、图4、原型图2	AC-05
US-04	对局后查看关键失误	复盘服务、分析节点数据、结果页	图3、原型图3	AC-04
US-05	导入固定局面和导出摘要	FileRepository、导入/导出接口	表4、图5	UC-07
US-06	AI 不可用时继续手动分析	超时处理、模板解释、离线状态	图1、图4	AC-03
FR-01/02	棋盘交互、规则校验	BoardView、RuleService、BoardState	图2、图3	AC-01/02
FR-03/04/05	难度、AI 应答、评价展示	GameService、EngineAdapter	图2、表4	AC-01/AI-01
FR-06/07/08	解释、复盘、保存导出	ExplanationService、ReviewService、FileRepository	图2、图3、表4	AC-04/05
性能/可靠性/隐私	实验2非功能需求	超时边界、故障恢复、本地默认存储	ADR-02、图1	AC-03

2.1.2 普通程序、AI 和工具的边界

参与方	负责内容	不负责内容	结果检查者
用户	摆放局面、选择难度、落子、查看和确认解释	不需要理解引擎协议	用户确认多解或专业解释
普通程序	棋盘状态、规则合法性、胜负判断、保存、错误处理、界面状态	不凭规则代码判断复杂中局优劣	RuleService、Schema 校验
本地象棋引擎	候选着法、评价值、主变化	不生成教学措辞，不决定界面最终文案	EngineAdapter + 规则模块
模板/可选 LLM	将结构化局面事实整理为短解释和练习建议	不判定合法性，不产生棋步真值	棋子/着法一致性校验 + 人工确认
文件系统	保存用户主动选择的棋谱和本地 JSON	不自动上传网络	FileRepository 回读校验

三、2.2 技术选型与架构决策

3.2.1 候选方案比较

方案	熟悉程度	实现成本	AI/引擎接入	测试与打包	结论
Python + PySide6 + JSON	较高，适合个人快速开发	低	可用子进程调用本地引擎，也可接 HTTP	pytest、PyInstaller 生态成熟	选用
C# + WPF + SQLite	中等	中	桌面 UI 和本地数据库强	Windows 打包好，但学习成本更高	作为后备方案
Electron + TypeScript	中等	高	接口灵活	打包体积和前端工程量偏大	不作为 MVP 首选

3.2.2 最终技术选择

- ☐ 语言与界面：Python 3.11 + PySide6。原因是能快速实现桌面窗口、信号槽和跨模块测试。
- ☐ 体系结构：轻量分层 + Ports and Adapters。界面层不直接调用引擎；领域层不依赖具体 UI；引擎、解释器和文件存储都通过适配器替换。
- ☐ 数据保存：MVP 使用本地 JSON 文件，按 game_id 保存一局完整记录；后续数据量明显增大时再迁移 SQLite。
- ☐ AI 接入：优先本地中国象棋引擎进程；解释默认模板化，LLM 仅作为可选适配器，不作为规则和胜负判定器。
- ☐ 打包：PyInstaller 生成 Windows 单文件或目录包；本地引擎作为可配置资源目录随包发布。

3.2.3 架构决策记录（ADR）

ADR-01：采用分层 + 端口适配器

- ☐ 问题：引擎协议和界面逻辑变化时，怎样不影响规则和复盘代码？
- ☐ 考虑方案：单文件脚本、MVC、分层 + 端口适配器。
- ☐ 选择：分层 + 端口适配器。
- ☐ 理由：规则可独立单元测试；引擎可用 Mock 替换；未来从本地引擎切换到其他引擎时影响较小。
- ☐ 不足：文件和接口数量增加，对个人项目有一定样板代码成本。

ADR-02：本地优先，AI 结果必须经过程序校验

- ☐ 问题：模型或引擎失败时，如何避免展示虚假评价并保护棋谱隐私？

☐ 考虑方案：全部使用云模型、全部使用本地引擎、引擎本地 + 解释可选云端。

☐ 选择：引擎本地 + 模板解释为默认，云端 LLM 为可选。

☐ 理由：断网可用、费用可控、棋谱默认不上传；合法性和真值留在确定性代码中。

☐ 不足：本地引擎安装和性能差异需要在设置页提示；生成式解释质量有限。

ADR-03：MVP 使用 JSON，SQLite 作为替代方案

☐ 问题：如何在开发初期简单保存并调试对局？

☐ 考虑方案：纯内存、JSON、SQLite。

☐ 选择：JSON 文件，接口保持 Repository 抽象。

☐ 理由：无需数据库安装，便于查看、备份和测试；数据量小且为单用户。

☐ 替代方案：若保存超过 20 局、出现查询性能或并发需求，迁移 SQLite，不改变上层接口。

四、2.3 软件架构与模块设计

4.1 软件与外部环境关系图

图1 软件上下文图。图中浅蓝节点属于本软件，灰色节点为用户或外部依赖。MVP
默认只调用本地引擎和用户主动选择的文件；云端 LLM 是可选外部服务，发送前必须提示用户。

源文件：实验3设计文件/diagrams/01-系统上下文图.mmd

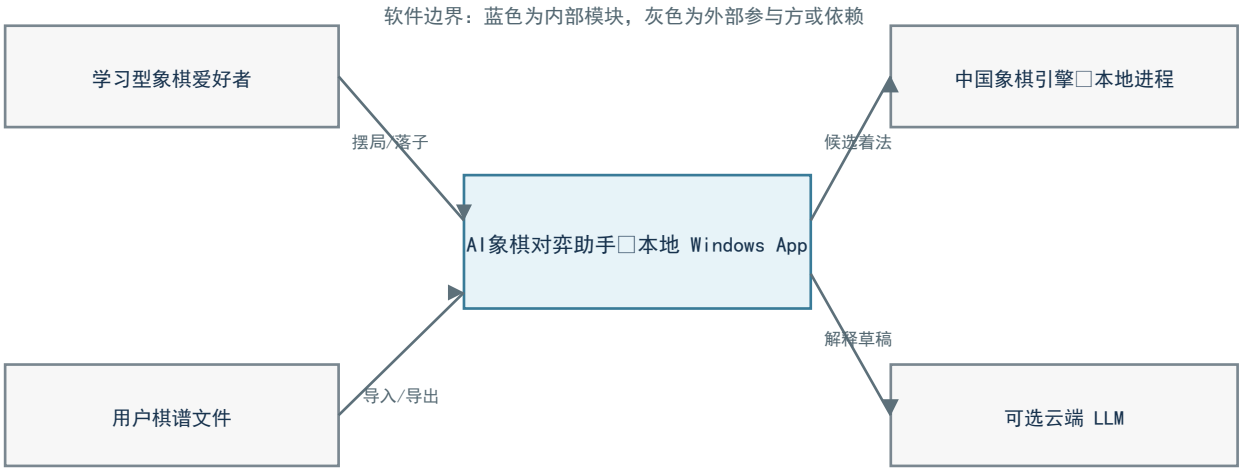


图1 软件上下文图

4.2 总体架构图和模块职责

图2 总体架构图。采用轻量分层结构：界面层只呈现状态和发送用户命令；应用服务层编排一次对局；领域层负责规则和核心对象；端口适配器隔离引擎、解释器和文件系统。

源文件：实验3设计文件/diagrams/02-总体架构图.mmd

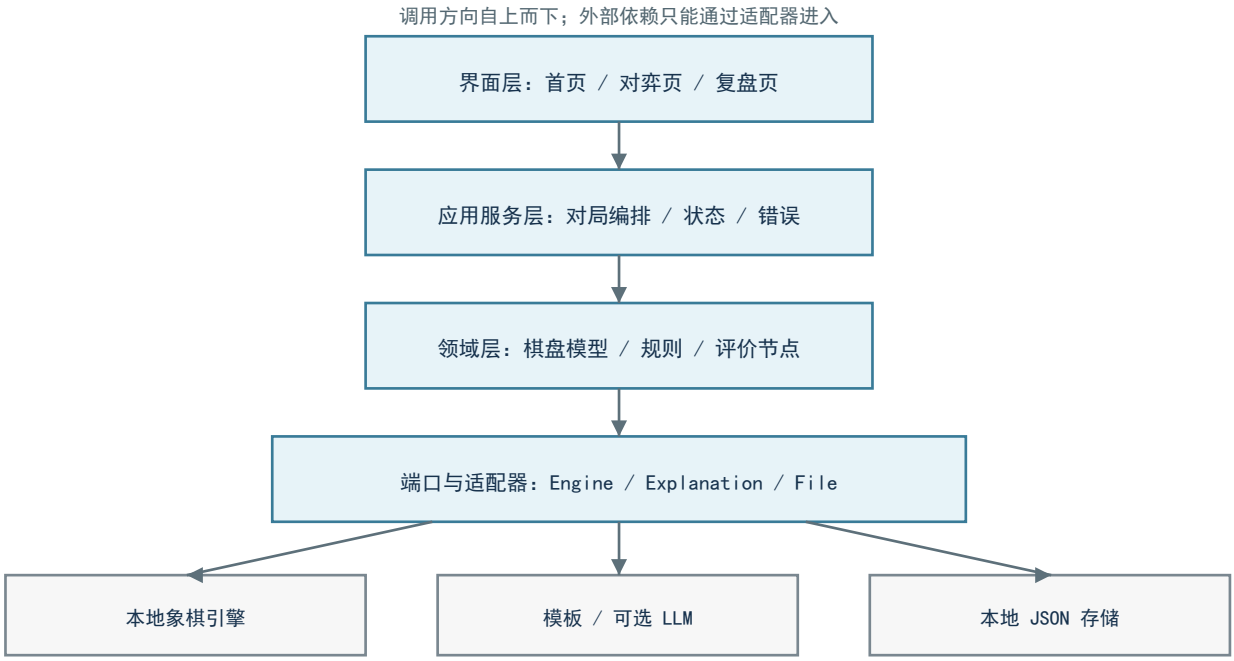


图2 总体架构图

模块职责：

模块	主要职责	允许依赖	禁止依赖
BoardView / 页面控制器	显示棋盘、按钮、状态、错误和加载态	ApplicationService	具体引擎进程
GameService	编排开始、落子、AI 回合和结束流程	Domain、Ports	具体 UI 控件
RuleService	检查棋子移动、将军、将死和局面完整性	BoardState	LLM
EngineAdapter	启动/停止引擎、解析候选着法、超时	引擎协议	页面控件
ExplanationAdapter	模板解释或可选 LLM 请求	结构化分析结果	规则真值
ReviewService	评价曲线、关键节点和练习建议	Move、AnalysisPoint	网络服务
FileRepository	JSON 保存、回读、导入导出	本地文件系统	远程账号系统

五、2.4 数据、接口与核心流程设计

5.1 数据模型

图3 数据模型图。 Game 是一次对局的聚合根，Move 保存用户和 AI 的每个半回合，AnalysisPoint 记录评价变化与人工确认标记，ReviewSummary 保存复盘结果。棋谱内容属于用户数据，默认只在本地 JSON 中保存；human_review_required 是安全边界字段，必须在 UI 中明确展示。

源文件：实验3设计文件/diagrams/03-数据模型图.mmd

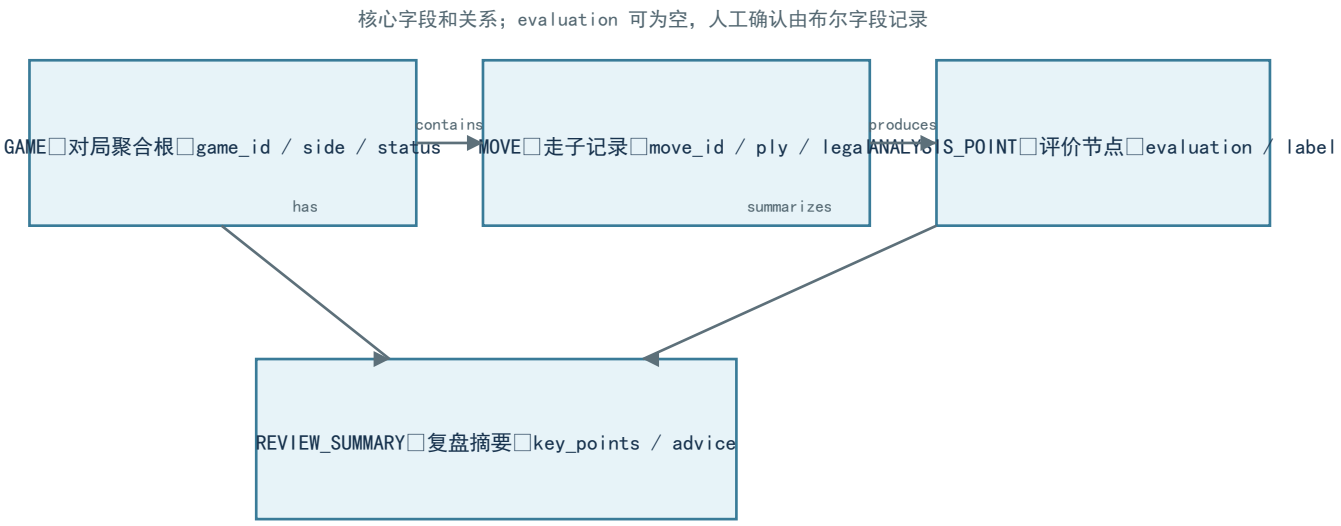


图3 数据模型图

主要约束：

- □ game_id、move_id、point_id、review_id 全局唯一；
- □ ply 从 1 开始递增，legal=false 的尝试不得写入正式 Move；
- □ from_square 与 to_square 必须经过规则模块校验；
- □ evaluation 允许为空，表示引擎失败或尚未分析；
- □ human_review_required=true 时，解释页面必须显示人工确认提示；
- □ 保存失败时保留内存状态，不覆盖上一次可回读文件。

5.2 主要接口

表4 核心接口设计

接口	提供模块	调用方	输入	输出	失败处理
POST /games（本地服务语义）	GameService	首页/对弈页	side、difficulty、initial_position	game_id、board_state	参数不完整返回 INVALID_GAME
POST /games/{id}/moves	GameService	对弈页	from、to、expected_turn	move_id、legal、next_turn	非法走法不写入 Move，返回 ILLEGAL_MOVE
POST /games/{id}/analysis	EngineAdapter	GameService	board_state、time_limit	candidates、evaluation、pv	超时返回 ENGINE_TIMEOUT，允许重试
POST /games/{id}/explanation	ExplanationAdapter	对弈页/复盘页	structured_facts	text、labels、human_review_required	LLM 失败切换模板
POST /games/{id}/finish	ReviewService	对弈页	moves、analysis_points	key_points、advice	部分分析也可生成，标记 partial=true
GET/PUT /games/{id}	FileRepository	首页/复盘页	game JSON	saved game / status	磁盘失败提示并保留内存数据

正常调用示例：POST /games/g-001/analysis 输入合法局面、执红和 3000 毫秒上限；输出 1—3 个候选着法、评价值和主变化。

错误调用示例：缺少将帅时返回 INVALID_POSITION，提示补充或重新摆局，engine_called=false。

5.3 核心流程图

图4 核心流程图。正常路径是“合法局面—合法落子—引擎应答—解释—结束复盘”；非法局面、非法走法和引擎失败都回到可恢复状态；多解或长变例会显式进入人工确认分支。

源文件：实验3设计文件/diagrams/04-核心流程图.mmd

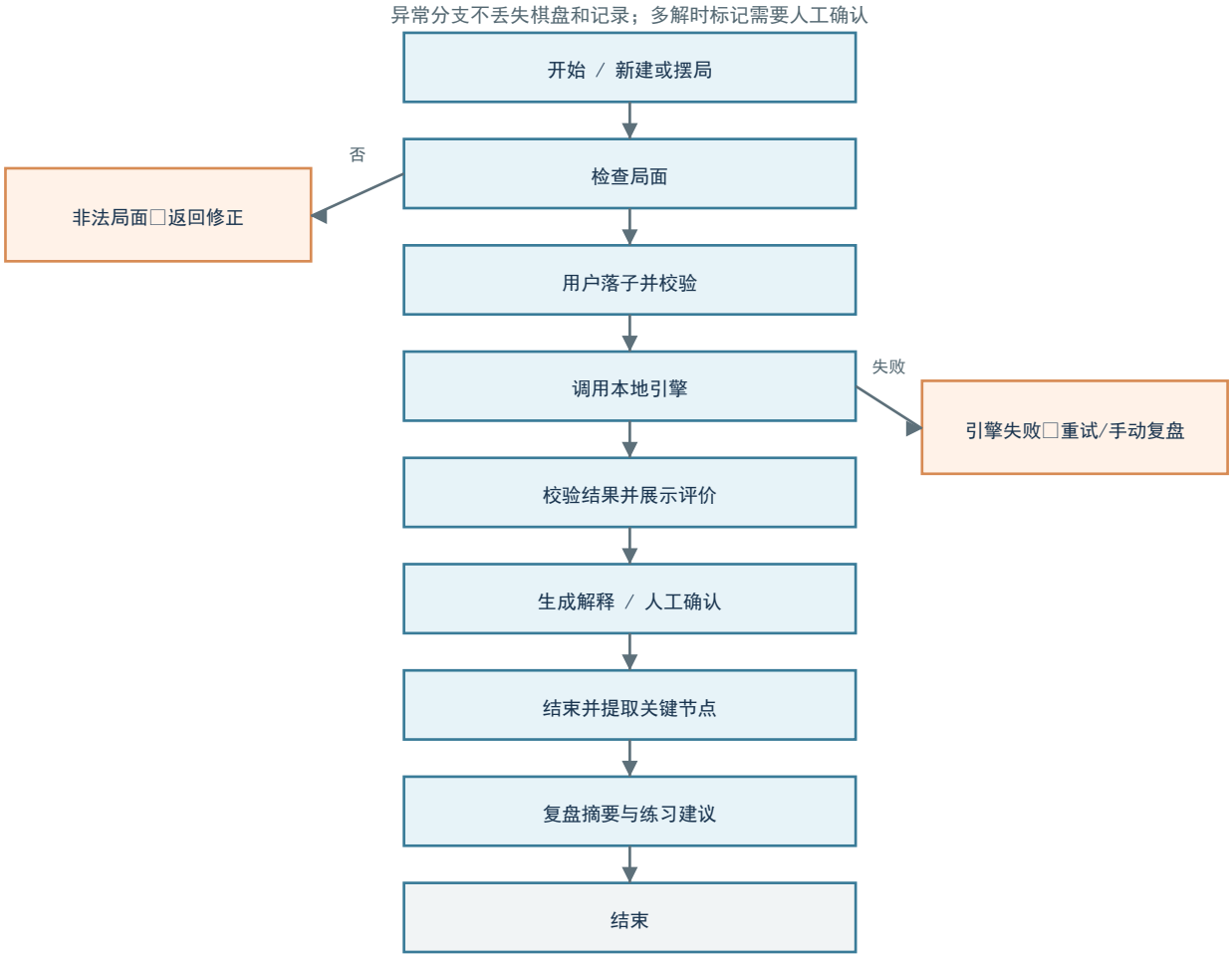


图4 核心流程图

六、2.5 界面原型与交互设计

6.1 页面导航

图5 页面导航图。首页是入口和最近对局列表；对弈页承载主要任务；复盘页提供关键节点、建议和导出；设置页只处理引擎状态和隐私选项。每个页面都提供返回、退出或再次操作路径。

源文件：实验3设计文件/diagrams/05-页面导航图.mmd

入口、核心任务和结果反馈形成闭环

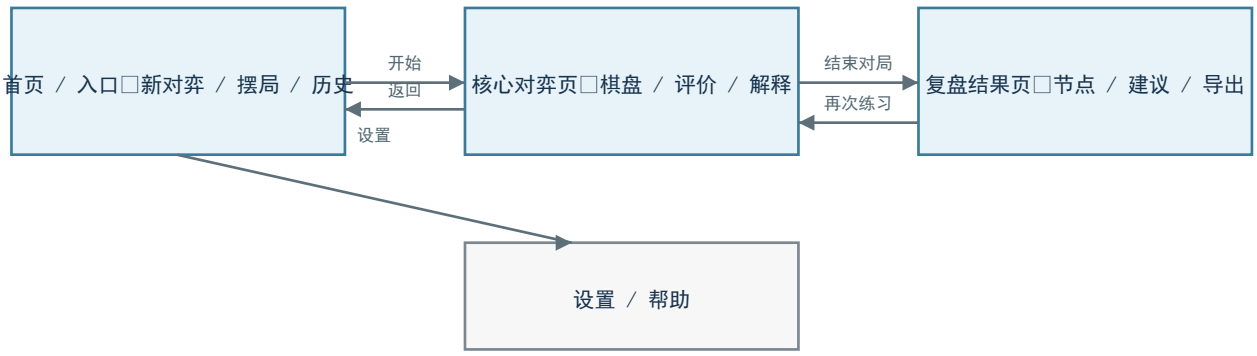


图5 页面导航图

6.2 关键界面原型

原型图采用桌面窗口比例，标注了静态信息、用户输入、用户命令和反馈状态。SVG 是可继续修改的源文件，提交到仓库后可用浏览器直接预览。

图6 首页/入口页原型

文件：实验3设计文件/prototypes/01-首页原型.svg



图6 首页/入口页原型

主要元素：开始新对弈、摆放局面、最近对局、设置；首次使用显示空数据提示。

图7 核心对弈页原型

文件：实验3设计文件/prototypes/02-对弈页原型.svg



图7 核心对弈页原型

主要元素：9×10 棋盘、执棋方、难度、当前状态、候选着法、评价、简短解释、悔棋和结束对局。AI 思考时在当前状态显示加载；引擎失败时替换为错误提示和重试/手动复盘。

图8 复盘结果页原型

文件：实验3设计文件/prototypes/03-复盘页原型.svg



图8 复盘结果页原型

主要元素：关键节点列表、节点详情、主变化、练习建议、再次练习和导出摘要；多解节点显示“需要人工确认”。

6.3 界面状态清单

状态	触发	界面反馈	用户下一步
空数据	首次打开或没有历史对局	显示空状态说明和开始按钮	新建对弈/摆局
加载中	请求引擎或解释器	禁止重复提交，显示思考状态	等待、取消或重试
非法输入	局面或走法不符合规则	标出错误格、错误码和修正建议	修正或返回
引擎失败	超时、进程异常、结果不可解析	保留棋盘和已走棋步，不展示虚假评价	重试或手动复盘
人工确认	多解、长变例或置信不足	显示候选列表和“需要人工确认”	选择练习方向或继续
成功完成	结束对局并生成摘要	展示关键节点和建议	再次练习、导出、返回

七、2.6 设计一致性检查与仓库版本证据

7.1 检查记录

检查项	检查依据	结果	调整
核心流程可从入口走到复盘	US-01、US-04、AC-01、AC-04	通过	首页、对弈页、复盘页形成闭环
非法局面不会调用引擎	US-02、AC-02	通过	RuleService 置于 EngineAdapter 之前
引擎失败不产生虚假评价	US-06、AC-03、ADR-02	通过	增加 ENGINE_TIMEOUT 和手动复盘状态
AI 解释可被程序和用户复核	US-03、AC-05	通过	增加 human_review_required 字段和 UI 标记
数据、接口和界面字段一致	FR-05/07/08	通过	evaluation、key_points、practice_advice 使用同名语义
隐私与离线要求落实	实验2非功能需求	通过	默认本地 JSON，不要求登录；云解释为可选

7.2 仓库路径与 Commit 证据

下表是提交到个人 GitHub/Gitee 后应填写的版本证据。由于当前工作区不是你的远程个人仓库，Commit 需要你按附录命令在自己的仓库中生成；不能凭空填写不存在的远程 Commit。

内容	建议仓库路径	对应 Commit
实验2需求 V1	docs/experiments/exp2-requirements-v1.md	提交后填写 git log -1 --oneline
实验3设计报告	docs/experiments/exp3-architecture-ui-v1.md	与本次设计文件同一次提交
架构、数据、流程、导航图源文件	design/diagrams/	同上
首页、对弈页、复盘页原型	design/prototypes/	同上
PDF 报告（若老师要求仓库存档）	docs/reports/实验3-软件架构与界面设计.pdf	同上

八、操作记录

1. 需求输入：从实验2提取 US、FR、AC 编号，限定一条核心用户流程。
2. 技术分析：比较 Python + PySide6、C# + WPF、Electron + TypeScript，选择个人实现成本更低且方便打包的 Python + PySide6。
3. 架构取舍：用分层 + 端口适配器隔离引擎和解释器；用 ADR 记录本地优先、JSON 保存和替代方案。
4. 设计检查：逐项检查需求、架构模块、数据字段、接口、界面和验收条件，记录了引擎失败、人工确认和隐私边界。
5. 文件保存：图源文件放入 design/diagrams/，界面原型放入 design/prototypes/，报告和实验2需求说明放入 docs/experiments/；使用附录命令提交并记录 Commit。

九、实验小结

本次实验将实验2的产品需求转换为可指导开发的设计基线。系统采用轻量分层 + 端口适配器，普通程序负责规则、状态、保存和错误处理，本地象棋引擎负责候选着法和评价，模板或可选 LLM 只负责解释草稿，并且所有结果都要经过程序校验或用户确认。

数据设计以 Game 为聚合根，围绕 Move、AnalysisPoint 和 ReviewSummary 保存一次对局的完整学习反馈；接口设计覆盖开始、落子、分析、解释、结束复盘和本地读写。三个关键原型分别覆盖入口、核心任务和结果反馈，并明确空数据、加载、错误、人工确认和重新操作状态。通过需求—设计追踪和一致性检查，确保 US、FR、AC 能够追溯到模块、数据、接口和界面。

附录：GitHub/Gitee 提交步骤

1. 在 GitHub 或 Gitee 创建一个空仓库，例如 AI-Xiangqi-Assistant。
2. 将实验2需求文件复制为 docs/experiments/exp2-requirements-v1.md，将本报告复制为 docs/experiments/exp3-architecture-ui-v1.md。
3. 将 实验3设计文件/diagrams/ 复制为 design/diagrams/，将 实验3设计文件/prototypes/ 复制为 design/prototypes/。
4. 在仓库根目录执行以下命令：

```
git init

git branch -M main

git add docs design README.md

git commit -m "docs: add experiment 3 architecture and UI design"
```

```
git remote add origin <你的仓库地址>

git push -u origin main

git log -1 --oneline

git rev-parse HEAD
```

5. 在仓库网页检查文件路径，复制仓库 URL、短 Commit 和完整 Commit，填回本报告的“仓库路径与 Commit 证据”表。